

Technical Research & Development Document

Assignment 2 | Project & Portfolio IV | R&D Demonstrator

AI-Powered American House Price Dashboard

Technical proof-of-concept for a housing market dashboard that connects CSV data loading, data cleaning, market filters, interactive charts, regression model comparison, and a user-driven fair value prediction workflow.

Student	Eric Reyes
Course	Project & Portfolio IV
Assignment	Assignment 2 - Technical Research & Development Project
Date	June 2026

R&D proof focus: dataset pipeline, visualization, model evaluation, and prediction interaction.

Executive Summary

HomeScope is a housing market dashboard that uses local listing data, national trend data, data cleaning, visual analytics, and regression modeling to help a user understand whether a listing appears low, fair, or above market. The Assignment 2 demonstrator proves that the selected technology stack can operate interdependently: Streamlit provides the interactive interface, Pandas loads and prepares data, Plotly displays charts, Scikit-learn trains and evaluates models, and the best trained model powers a user-driven price prediction workflow.

The current demonstrator uses the American Housing dataset as the primary data source and ASPUS as a supplemental national trend dataset. The housing dataset contains 39,981 rows and 14 columns, including price, beds, baths, living space, city, state, county, demographic fields, and geographic coordinates. The ASPUS dataset contains 253 quarterly observations of U.S. average sales price data. This amount of data is enough for a credible R&D proof because the application can filter listings, compute market metrics, build visualizations, train models, compare model performance, and make a sample prediction through the interface.

Technology Stack Researched

The R&D stack was intentionally reduced from the final full-stack idea to a Streamlit-based demonstrator. This decision lowered delivery risk while still proving the important technical chain. Streamlit supports a fast browser-based interface with widgets, charts, forms, buttons, and page sections. Its caching system supports separate caching for data outputs and resource-style objects, which is useful because the app reloads cleaned data and reuses trained model results rather than recomputing every time a user changes a widget (Streamlit, n.d.).

Pandas is used for CSV loading and tabular transformation. The official pandas API supports reading CSV files directly into DataFrames, which fits HomeScope because both the listing dataset and the ASPUS trend dataset are local CSV files (pandas development team, n.d.). Plotly Express is used for quick interactive charts, including histograms, scatter plots, bar charts, line charts, and residual plots. Scikit-learn is used for the regression pipeline, including the baseline Linear Regression model, the Random Forest Regressor, and model metrics. Random Forest Regressor is especially useful for this project because it averages multiple decision tree regressors and can capture non-linear relationships between location, living space, demographics, and price (scikit-learn developers, n.d.-a).

ASPUS comes from the Federal Reserve Bank of St. Louis FRED database and represents the average sales price of houses sold for the United States. The series is quarterly, measured in dollars, and not seasonally adjusted. In this project, it is used as national trend context instead of as a direct prediction feature because the local listing dataset is cross-sectional while ASPUS is time-series data (Federal Reserve Bank of St. Louis, 2026).

Why This Option Over Other Options

The original pitch included a future React or Next.js frontend with a Python FastAPI backend. That approach remains appropriate for the final PP4 direction, but it introduces extra integration work before the data and model pipeline has been proven. The R&D assignment requires evidence that the technologies work together and that user-driven interactions are meaningful. Streamlit provides that proof faster because the same Python application can load data, transform data, train models, and serve a browser interface. This makes it easier to focus on the unique HomeScope interactions rather than spending time on routing, frontend state, API deployment, and authentication.

The R&D proof also avoids unnecessary features. Login screens, settings pages, and dark mode do not prove the housing model, data pipeline, or market comparison logic. The demonstrator instead focuses on dataset loading, cleaning, filters, charts, model comparison, residual analysis, and prediction output. This scope is better aligned with a one-month student project because it prioritizes the parts that make HomeScope different from generic dashboard projects.

PP4 Feature List

1. Load and clean the American Housing dataset.
2. Load ASPUS national trend data for market context.
3. Create derived features such as price per square foot and location/demographic modeling fields.
4. Filter listings by state, city, beds, baths, and living space.
5. Display market snapshot metrics: matching listings, average price, median price, and average price per square foot.
6. Display visualizations for price distribution, price versus living space, city/state average prices, national price trend, and residuals.
7. Train at least two models: Linear Regression and Random Forest Regressor.
8. Evaluate models with MAE, RMSE, and R-squared.
9. Select the best model by MAE for prediction.
10. Provide a user-driven prediction form.
11. Display predicted fair value, listing price, market label, and prediction comparison chart.
12. Organize code into reusable data and model modules.
13. Document model limitations and error patterns.
14. Stretch: add map view, feature importance, saved comparisons, and more model experimentation.

R&D Feature List

Area	R&D Feature
User Interface	Streamlit title, captions, sections, sidebar filters, prediction form, and button-driven output.
Data	Load American_Housing_Data_20231209.csv and ASPUS.csv from the project data folder.
Cleaning	Remove invalid or missing modeling values, create

	derived price per square foot, and prepare model inputs.
Filtering	Apply user-selected state, city, beds, baths, and living space filters to update metrics and charts.
Visualization	Show price distribution, price versus living space, city average prices, ASPUS trend line, residual plot, and prediction comparison chart.
Machine Learning	Train Linear Regression and Random Forest Regressor, compare MAE, RMSE, and R-squared, then select the best model by MAE.
Prediction	Use the selected best pipeline to estimate fair value from the user-selected property inputs.
Hardware	No special hardware integration is required because the application is browser-based and uses local files.

Hands-On Development Results

The demonstrator successfully connects the main parts of the technology chain. The sidebar filter controls update the selected market. The market snapshot computes listing count, average price, median price, and price per square foot from filtered records. The visualization layer shows price distribution and the relationship between living space and price. The city average price chart gives local comparison context, while the ASPUS line chart provides national historical context.

The model training section compares Linear Regression and Random Forest Regressor. In the observed run, Linear Regression produced an MAE of about \$190,035, RMSE of about \$298,236, and R-squared of 0.750. Random Forest Regressor produced an MAE of about \$147,247, RMSE of about \$275,123, and R-squared of 0.792. Random Forest was therefore selected as the best model by MAE. Mean absolute error is useful in the demo because it is readable as an average dollar error, and the official scikit-learn metric defines its best possible value as 0.0 (scikit-learn developers, n.d.-b).

The prediction section accepts a sample property through the interface and uses selected location medians for population, density, income, latitude, and longitude. This design keeps the interface simple while still giving the model the location and demographic variables needed by the pipeline. The output shows predicted fair value, listing price, a market label, and a bar chart comparing the two values.

Triumphs

- The local data workflow is stable. Both housing and ASPUS CSV files load from the project folder without needing an external API during the demo.
- The interface supports meaningful interaction. A user can change filters, view updated charts, and submit a prediction form.
- The model comparison is visible and explainable. The table makes it clear why the Random Forest Regressor is selected over the baseline model.

- The app demonstrates technical depth without overbuilding. Data cleaning, visualization, model training, evaluation, and prediction are all represented in one app.

Roadblocks and Lessons Learned

The main roadblock is that housing price prediction is sensitive to outliers and location effects. Some properties are luxury listings or unusually priced records, which can stretch the distribution and make regression errors larger. The residual plot confirms that larger predicted values tend to create wider error spread. This means the final project should include stronger outlier handling, optional log-transformed prices, and possibly separate analysis by city or state.

A second roadblock is balancing user input simplicity against model feature requirements. The model uses variables such as zip code population, density, median household income, latitude, and longitude. Asking a regular user to enter every one of those values would make the interface frustrating. The R&D solution uses medians from the selected city to fill those fields automatically. This is a practical compromise for the demo, but the final version should explain these assumptions or expose them in an advanced section.

A third lesson is that the R&D prototype benefits from caching. Data loading and model training are expensive compared with simple widget changes. Streamlit caching separates data caching and long-lived resource caching, which makes the app feel more responsive during repeated interaction (Streamlit, n.d.).

Limitations, Risks, and Mitigation

Risk	Issue	Mitigation
Outliers and skewed prices	Luxury or unusual listings can distort averages and model training.	Cap or remove extreme outliers, compare raw and cleaned distributions, and consider log-price modeling.
Location bias	City and state effects can dominate price prediction.	Evaluate errors by city/state and consider localized models if needed.
Cross-sectional data	The primary dataset is listing-level data, not a live market feed.	Present the tool as a research dashboard and avoid claiming real-time appraisal accuracy.
ASPUS mismatch	ASPUS is national time-series data while the housing dataset is local listing data.	Use ASPUS only as trend context unless a time-aligned dataset is added later.
Model interpretability	Random Forest can perform better but is less transparent than Linear Regression.	Add feature importance and plain-language explanations in the final version.
Deployment size/performance	Large data files and model training can slow deployment.	Save a trained model pipeline, cache cleaned data, and limit chart sample sizes.

Feasibility Decision

The technology and the HomeScope pitch work together. The R&D application proves that the project is technically possible because the interface, local files, data cleaning, visualizations, model training, model comparison, and prediction workflow are already connected. The final production cycle should keep the same core pipeline and improve quality rather than adding unrelated features. The safest next step is to strengthen data cleaning, add clearer model documentation, and keep the final MVP focused on market averages, listing comparison, and fair value prediction.

References

Federal Reserve Bank of St. Louis. (2026). Average sales price of houses sold for the United States [ASPUS]. FRED. <https://fred.stlouisfed.org/series/ASPUS>

pandas development team. (n.d.). pandas.read_csv. pandas documentation. https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html

scikit-learn developers. (n.d.-a). RandomForestRegressor. scikit-learn documentation. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html>

scikit-learn developers. (n.d.-b). mean_absolute_error. scikit-learn documentation. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.mean_absolute_error.html

Streamlit. (n.d.). Caching overview. Streamlit documentation. <https://docs.streamlit.io/develop/concepts/architecture/caching>

AI Prompt Documentation. AI assistance was used for brainstorming, formatting support, and revision support. Prompts documented for this assignment included:

- "Generate an example feature list structure for a data-dashboard project that compares records, filters results, and summarizes key metrics."
- "Generate an example R&D feature list for a small demonstrator app that proves a user interface, CSV data processing, charts, and a regression model can work together."
- "Review this Feature Lists document for clarity, MVP focus, and alignment with the assignment requirements."
- "Generate an example technical R&D report outline for a software prototype that uses data cleaning, visualization, model evaluation, and user input."
- "Review this technical R&D document for organization, technical clarity, and APA-style source formatting."

These prompts were used as support for examples, planning, and revision. Final technical decisions were based on the working HomeScope R&D demo and the code review performed for this assignment.